МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 1

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Основы OpenMP: параллельные области, распределение работ, синхронизация потоков»

Работу выполнил
Студент гр. 4404
Зарипов А.Э.

Казань, 2026

1. Цель работы

Изучение основ параллельного программирования для систем с общей памятью: параллельные области и идентификация потоков, распределение итераций цикла между потоками (типы расписания schedule), примитивы синхронизации потоков. В ходе работы выполнены задачи 1–3 методических указаний «Параллельное программирование на OpenMP». Работа выполняется на языке Go, поэтому конструкции OpenMP моделируются стандартными средствами языка: горутины и runtime.GOMAXPROCS соответствуют команде потоков, sync.WaitGroup — неявному барьеру в конце параллельной области, sync.Mutex — критической секции, sync/atomic — атомарным операциям, каналы — средствам обмена и синхронизации.

2. Задание

Задача 1. Написать программу, где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Задача 2. Написать программу, в которой объявлен массив из 16000 элементов, инициализированный так, что значение элемента равно его порядковому номеру. Создать результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0. Запустить программу с 8 потоками при различных типах распределения работ.

Задача 3. Модифицировать задачу 1 так, чтобы потоки печатали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения; необходимо реализовать и продемонстрировать как можно больше из них.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком

решающие задачу; каждый запуск, о котором говорится в задаче (разное число потоков, разные типы распределения, разные версии программы), подтверждается скриншотом с видимой командой запуска и полным выводом.

3. Ход работы

3.1. Задача 1. Параллельная область и идентификация потоков

Программа считывает из командной строки число потоков N, ограничивает уровень параллелизма вызовом runtime.GOMAXPROCS(N) и создаёт N горутин (аналог команды потоков OpenMP). Каждая горутина печатает свой идентификатор, общее число потоков и строку «Hello World». Главная горутина ожидает завершения всех рабочих горутин с помощью sync.WaitGroup, что соответствует неявному барьеру на выходе из параллельной области. Мьютекс sync.Mutex используется только для того, чтобы строки разных потоков не перемешивались посимвольно; порядок вывода он не задаёт.

Фрагмент кода (lab1/task1/main.go):

```
for i := 0; i < numThreads; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        printMu.Lock()
        fmt.Printf("Thread %d of %d: Hello World\n", id, numThreads)
        printMu.Unlock()
    }(i)
}
wg.Wait()
```

Команды запуска:

```
cd lab1/task1
go run . 8
```

Ответ на вопрос задачи: вывод НЕ всегда идентичен. Потоки выполняются параллельно, и порядок, в котором они достигают оператора вывода, определяется планировщиком среды выполнения, поэтому от запуска к запуску порядок строк может меняться. Множество строк одинаково во всех запусках, но их последовательность программой не задаётся (синхронизация,

фиксирующая порядок, отсутствует). Для подтверждения выполнено три запуска программы с 8 потоками.

На рисунке 1 показан первый запуск программы с 8 потоками. Видна команда запуска и полный вывод: каждый из 8 потоков напечатал свой идентификатор, общее число потоков и строку «Hello World».

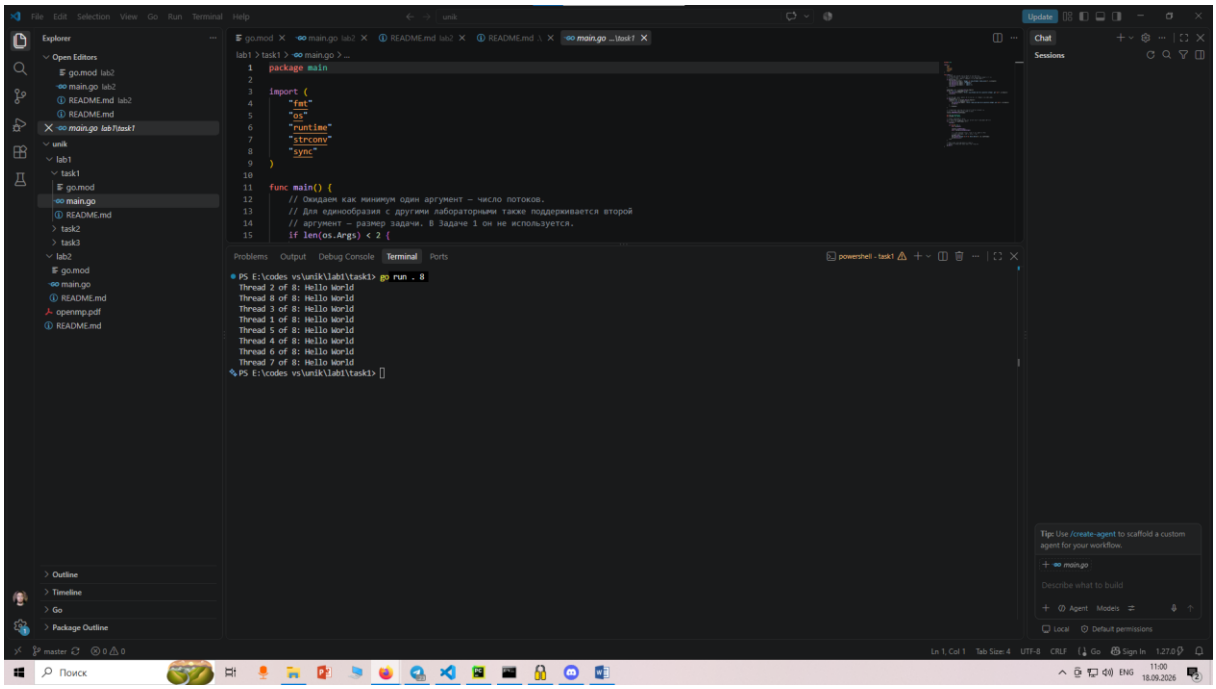


Рисунок 1 — Задача 1: запуск с 8 потоками, запуск № 1

На рисунке 2 показан второй запуск той же команды. Порядок строк может отличаться от рисунка 1, так как порядок выполнения потоков не определён. Если порядок совпал, это не гарантирует детерминированность вывода — программа порядок не фиксирует.

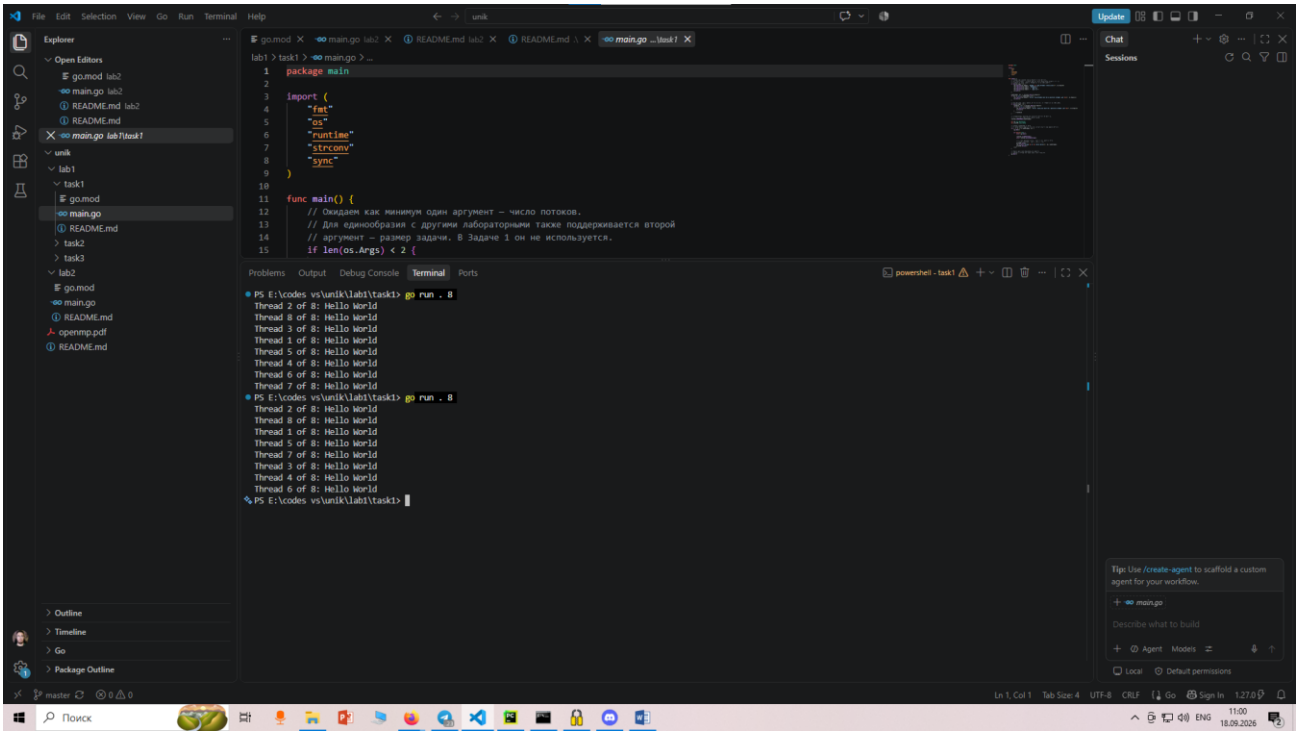


Рисунок 2 — Задача 1: запуск с 8 потоками, запуск № 2

На рисунке 3 показан третий запуск. Сравнение рисунков 1–3 показывает, что набор строк вывода во всех запусках одинаков, а их порядок может изменяться, что подтверждает ответ на вопрос задачи.

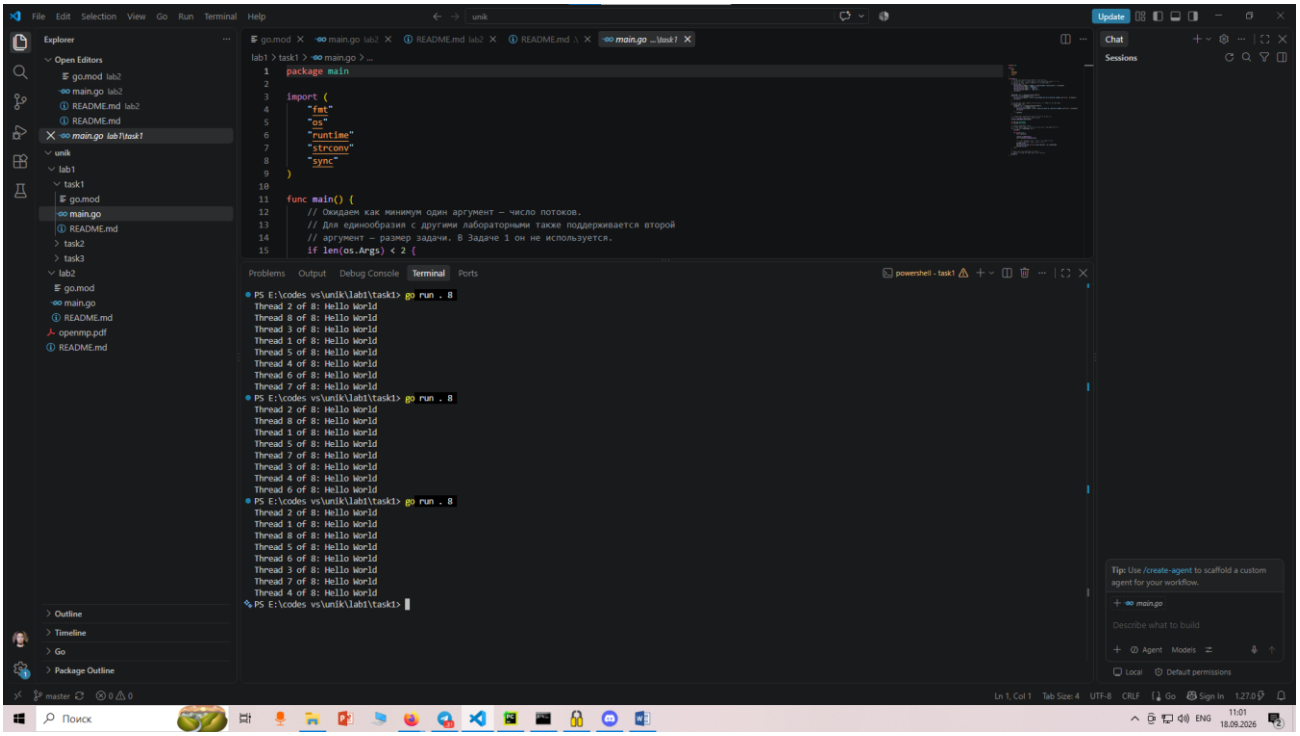


Рисунок 3 — Задача 1: запуск с 8 потоками, запуск № 3

3.2. Задача 2. Обработка массива при различных типах распределения работ

Программа создаёт массив a из N элементов (N = 16000 передаётся через командную строку), где a[i] = i, и вычисляет результирующий массив b по формуле b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0 для внутренних индексов i = 1 … N-2. Крайние элементы b[0] и b[N-1] в условии не определены и копируются из исходного массива. Обработка внутренних элементов распределяется между 8 потоками в соответствии с выбранным типом расписания, аналогичным условию schedule в OpenMP:

— static без блока: диапазон итераций делится между потоками непрерывными равными частями;

— static с блоком chunk: блоки фиксированного размера раздаются потокам циклически;

— dynamic: потоки берут блоки из общей очереди (защищена мьютексом, аналог критической секции), размер блока по умолчанию 1;

— guided: размер выдаваемого блока уменьшается по мере обработки массива, но не меньше минимального chunk;

— runtime: тип расписания и размер блока читаются из переменной окружения OMP_SCHEDULE / LAB2_SCHEDULE.

Фрагмент кода (lab1/task2/main.go, выдача блока в dynamic/guided):

```
schedMu.Lock()
if next > endIdx { schedMu.Unlock(); break }
startIdx := next
blockEnd := minInt(startIdx+blockSize-1, endIdx)
next = blockEnd + 1
schedMu.Unlock()
processBlock(startIdx, blockEnd)  // b[i] = (a[i-1]+a[i]+a[i+1])/3.0
```

Гонки данных отсутствуют: каждая итерация записывает только в свой элемент b[i], массив a не изменяется. Поэтому результирующий массив и контрольная сумма не зависят от типа расписания (при a[i] = i получаем b[i] = i, сумма равна N·(N-1)/2 = 127992000, программа выводит OK: true). Различаются лишь статистика распределения итераций и блоков по потокам и время выполнения.

Ниже приведены запуски программы с 8 потоками при различных типах распределения работ.

```
cd lab1/task2
go run . 8 16000 static
go run . 8 16000 static 1000
go run . 8 16000 dynamic 100
go run . 8 16000 guided 100
LAB2_SCHEDULE=dynamic,100 go run . 8 16000 runtime
```

На рисунке 4 показан запуск с расписанием static без задания блока: 15998 внутренних итераций разделены между 8 потоками непрерывными участками (6 потоков по 2000 итераций, 2 потока по 1999). Контрольная сумма совпала с ожидаемой (OK: true).

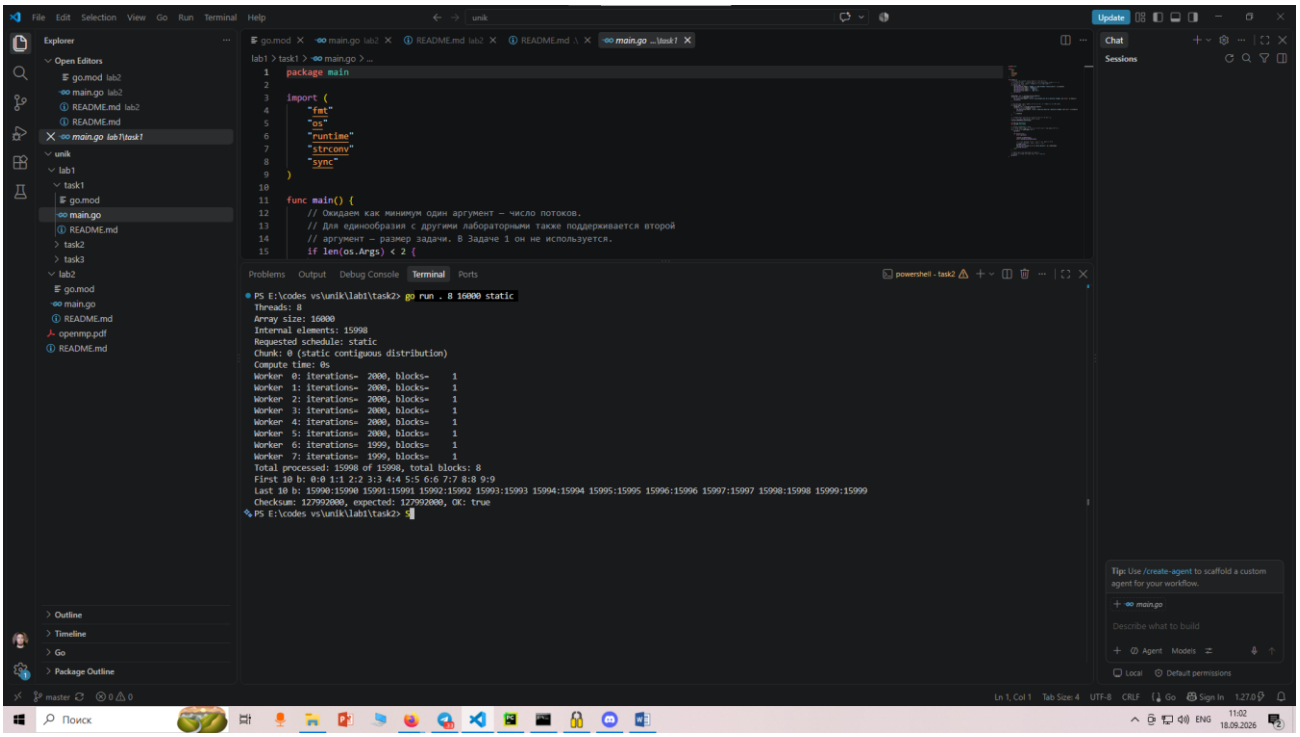


Рисунок 4 — Задача 2: 8 потоков, расписание static (без блока)

На рисунке 5 показан запуск с расписанием static и блоком 1000: итерации раздаются потокам циклически блоками по 1000 (каждый поток получает несколько блоков). Результат идентичен рисунку 4.

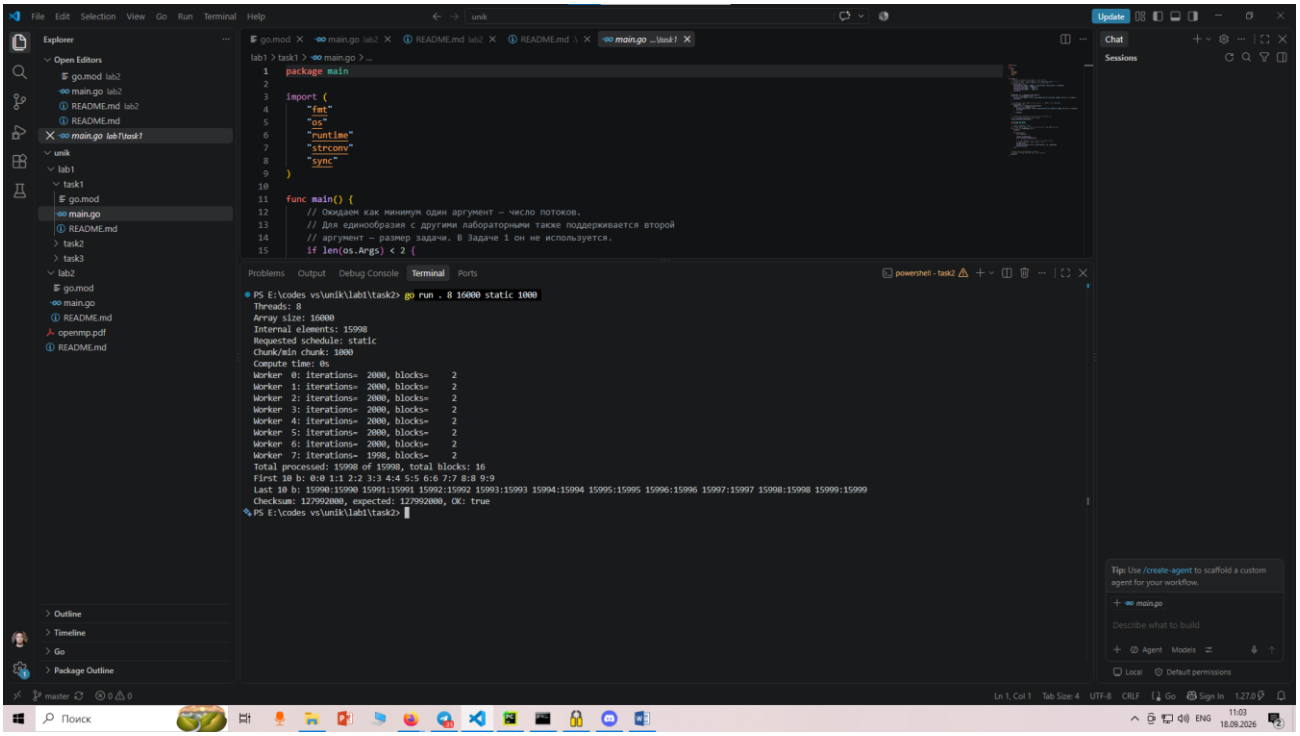


Рисунок 5 — Задача 2: 8 потоков, расписание static с блоком 1000

На рисунке 6 показан запуск с расписанием dynamic и блоком 100: потоки захватывают блоки из общей очереди по мере освобождения, поэтому число блоков у разных потоков может различаться. Все 15998 итераций обработаны, контрольная сумма совпала.

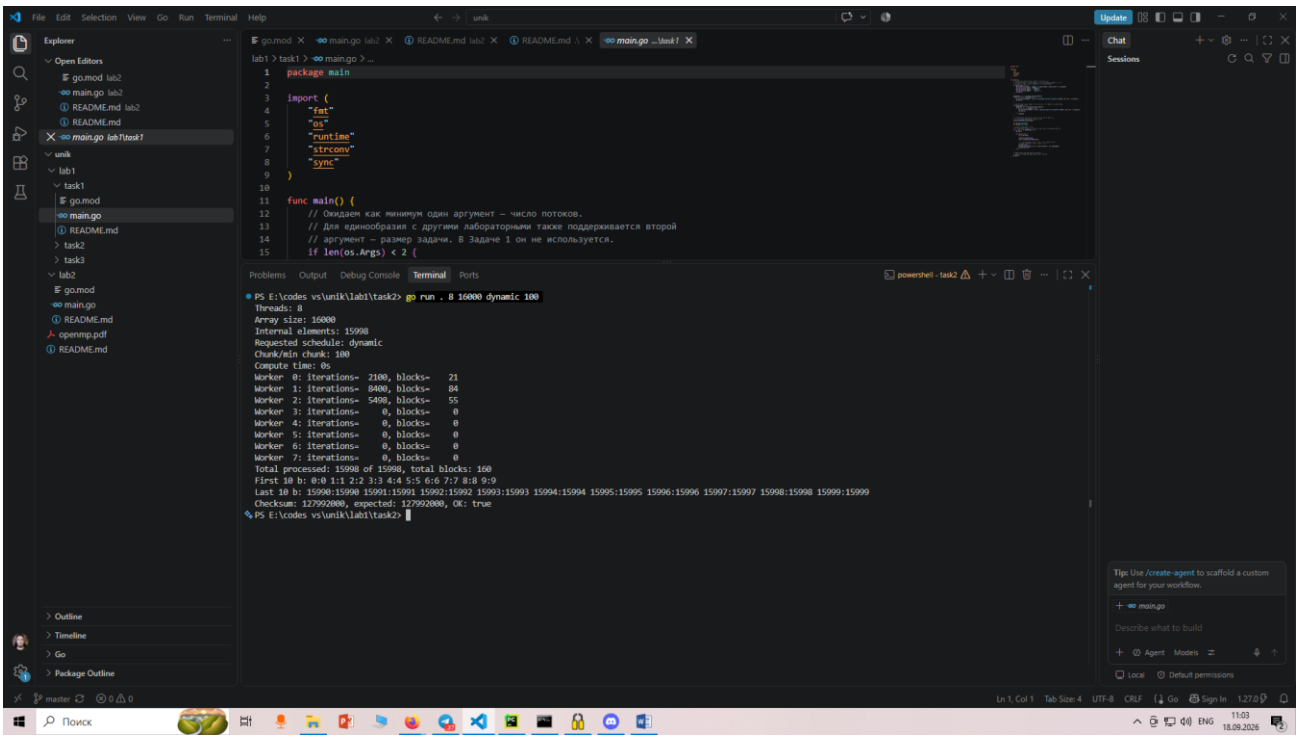


Рисунок 6 — Задача 2: 8 потоков, расписание dynamic с блоком 100

На рисунке 7 показан запуск с расписанием guided и минимальным блоком 100: размер выдаваемых блоков уменьшается по мере обработки массива, что позволяет балансировать загрузку при меньших накладных расходах. Результат совпадает с предыдущими запусками.

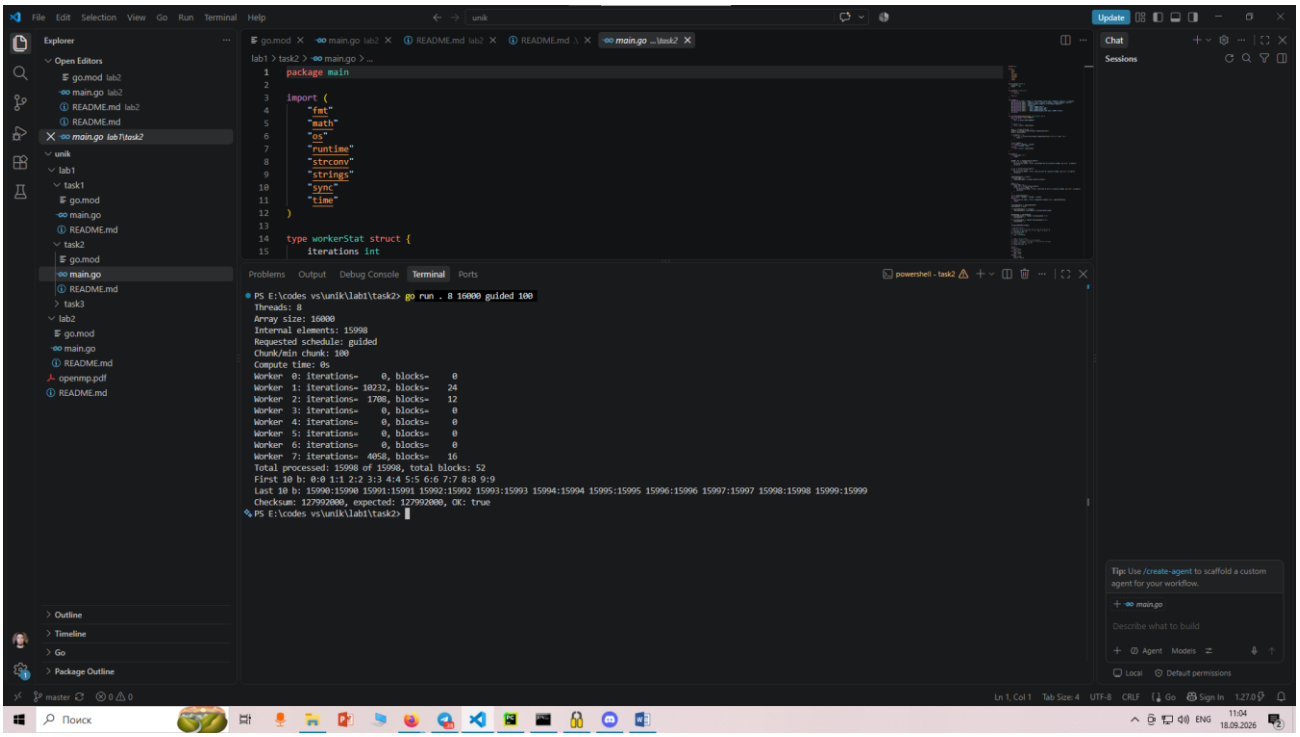


Рисунок 7 — Задача 2: 8 потоков, расписание guided с минимальным блоком 100

На рисунке 8 показан запуск с расписанием runtime: конкретный тип распределения (dynamic,100) взят из переменной окружения LAB2_SCHEDULE, что соответствует поведению OMP_SCHEDULE в OpenMP.

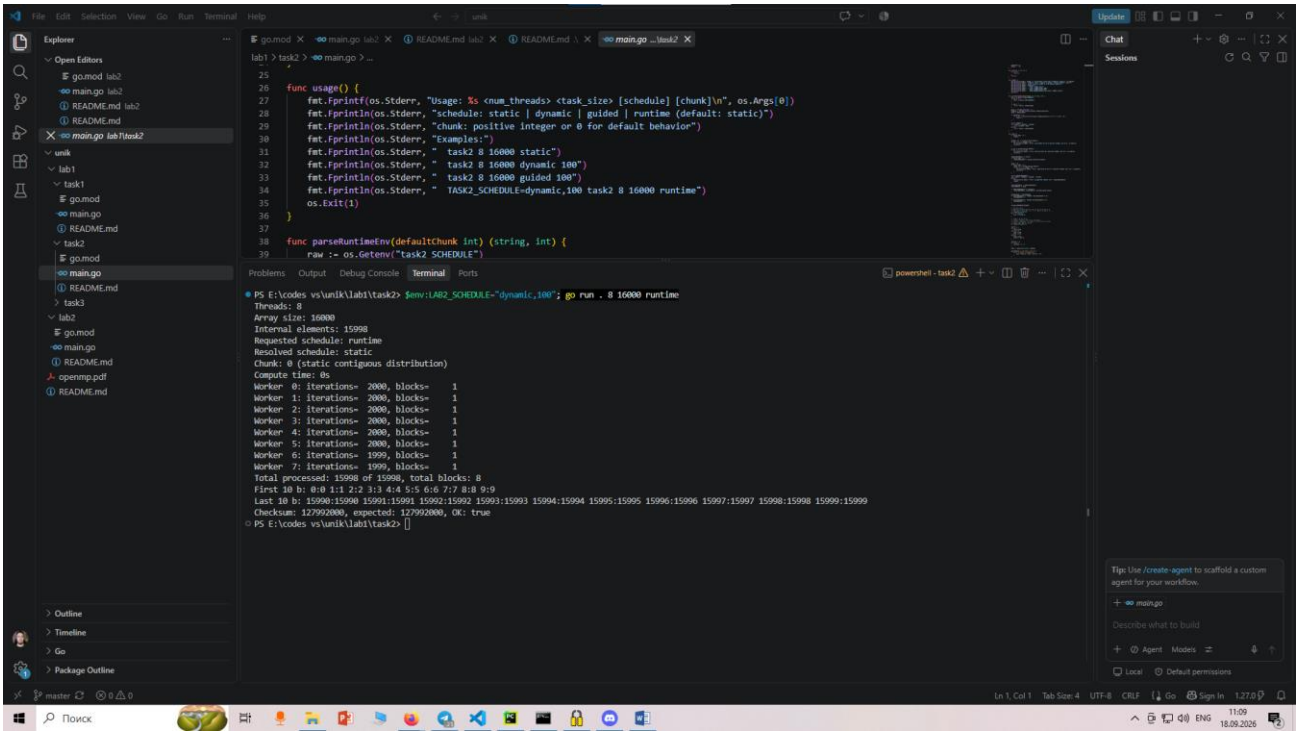


Рисунок 8 — Задача 2: 8 потоков, расписание runtime (dynamic,100 из окружения)

3.3. Задача 3. Вывод идентификаторов потоков в обратном порядке

Для фиксации порядка вывода необходимы средства синхронизации. В работе реализовано пять способов (разные версии программы выбираются аргументом командной строки):

1) активное ожидание (spinlock) на атомарном счётчике из пакета sync/atomic: поток крутится в цикле, пока счётчик не примет значение его идентификатора;

2) пассивное ожидание на условной переменной sync.Cond: потоки засыпают и пробуждаются сигналом, когда подходит их очередь;

3) цепочка каналов (передача эстафеты): поток i ожидает сигнал от потока i+1, печать начинается с потока N-1;

4) координатор с массивом каналов и рукопожатием: главный поток выдаёт разрешения на печать в обратном порядке и дожидается подтверждения каждого потока;

5) цепочка sync.WaitGroup: поток i ожидает завершения печати потока i+1 через wgs[i].Wait().

Фрагмент кода (lab1/task3/main.go, способ 2):

```
mu.Lock()
for current != id {
    cond.Wait()
}
fmt.Printf("Thread %d of %d: Hello World\n", id, n)
current--
cond.Broadcast()
mu.Unlock()
```

Команды запуска (по одному способу на запуск):

```
cd lab1/task3
go run . 8 1000 mutex
go run . 8 1000 cond
go run . 8 1000 chan_chain
go run . 8 1000 chan_array
go run . 8 1000 wg_chain
```

На рисунке 9 показана версия программы со способом 1 (активное ожидание на атомарном счётчике). Идентификаторы печатаются строго в порядке 7, 6, 5, 4, 3, 2, 1, 0, что подтверждает корректность способа.

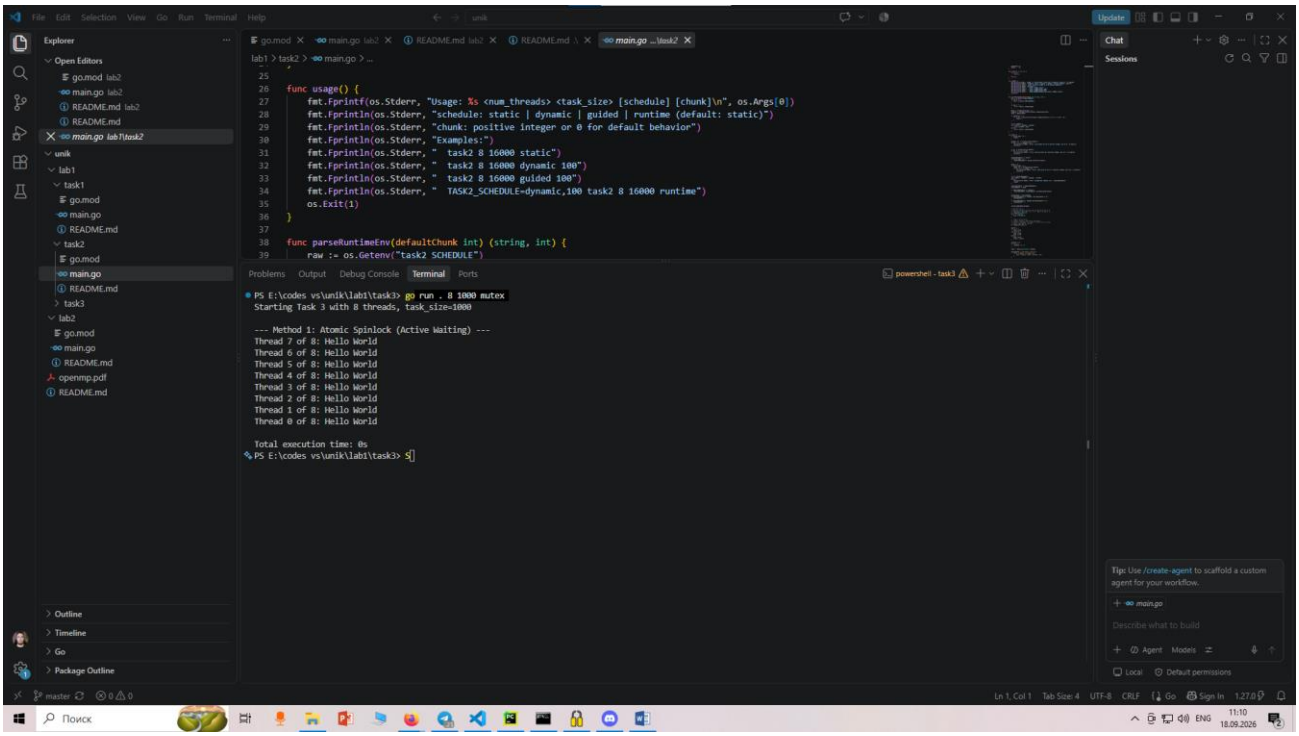


Рисунок 9 — Задача 3: способ 1 (активное ожидание, атомарный счётчик)

На рисунке 10 показана версия со способом 2 (условная переменная sync.Cond). Потоки ожидают своей очереди без нагрузки на процессор; порядок вывода также строго обратный.

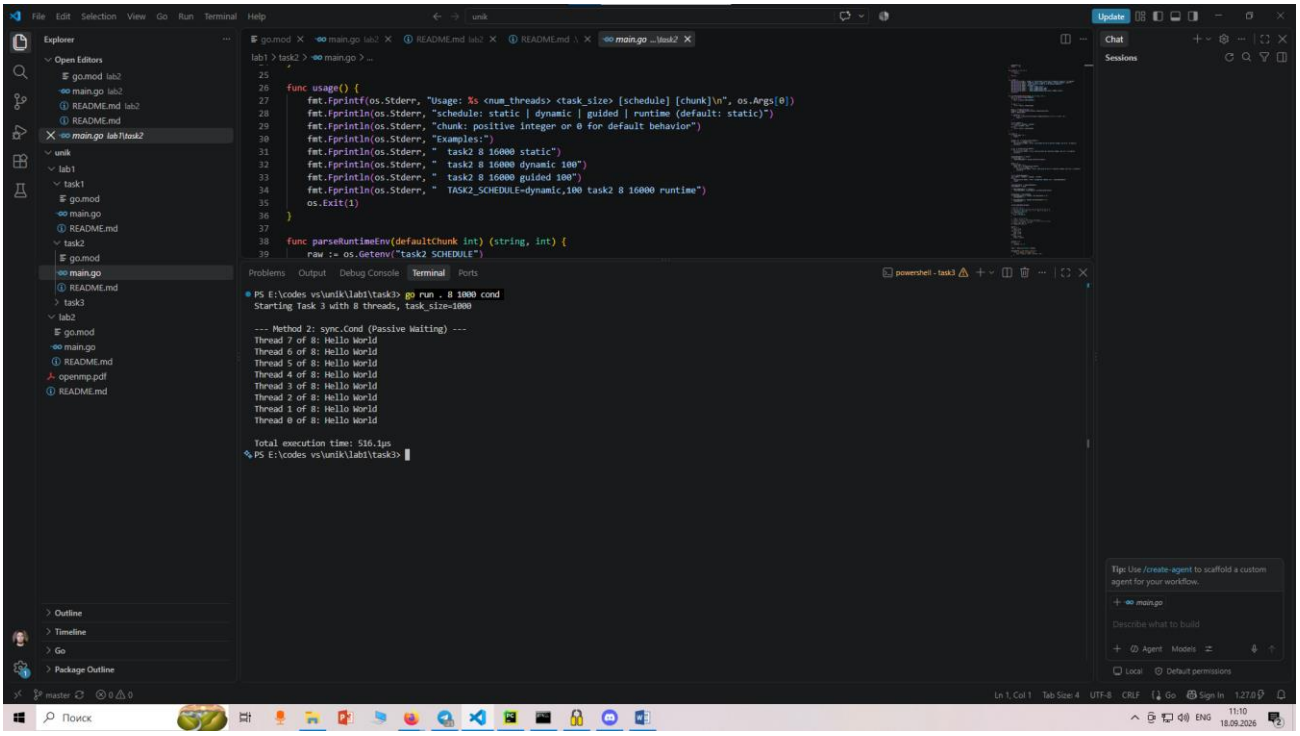


Рисунок 10 — Задача 3: способ 2 (условная переменная sync.Cond)

На рисунке 11 показана версия со способом 3 (цепочка каналов). Право печати передаётся от потока 7 к потоку 0; порядок вывода строго обратный.

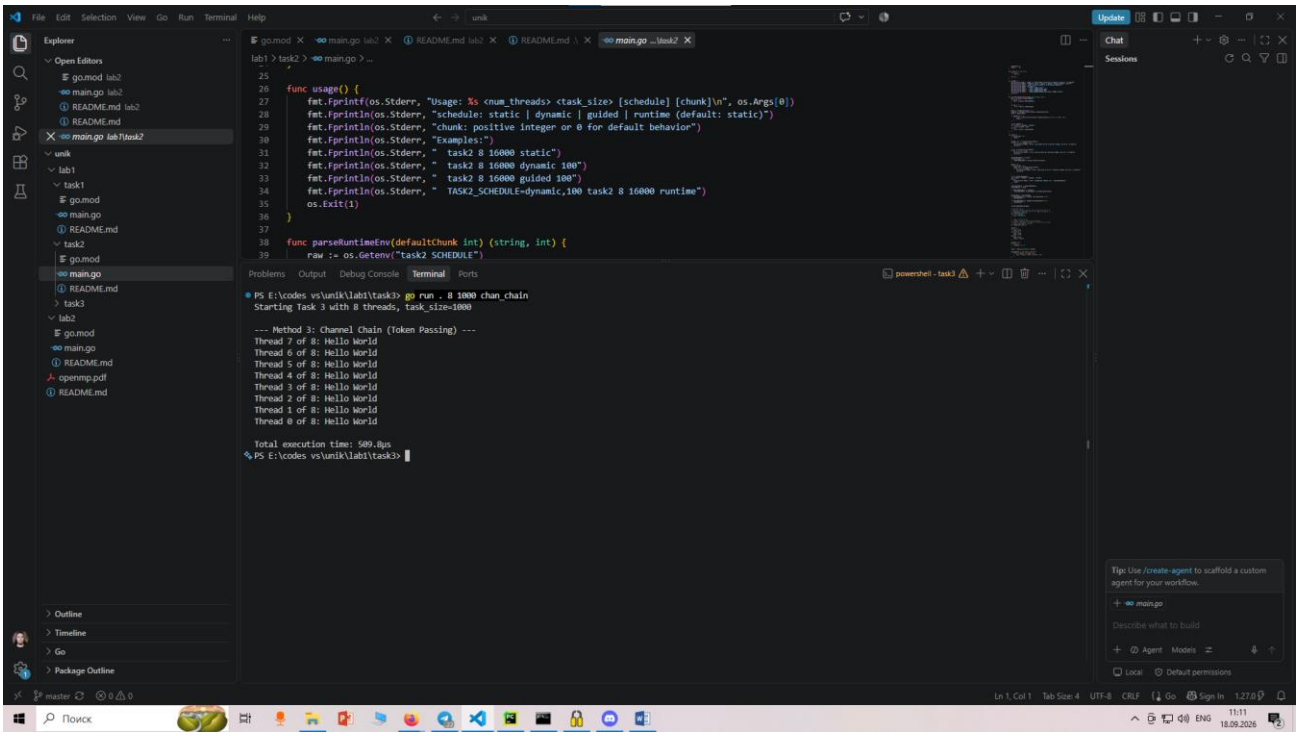


Рисунок 11 — Задача 3: способ 3 (цепочка каналов)

На рисунке 12 показана версия со способом 4 (координатор). Главный поток выдаёт разрешения в порядке 7, 6, …, 0 и дожидается подтверждения каждого потока — наиболее жёсткий контроль порядка.

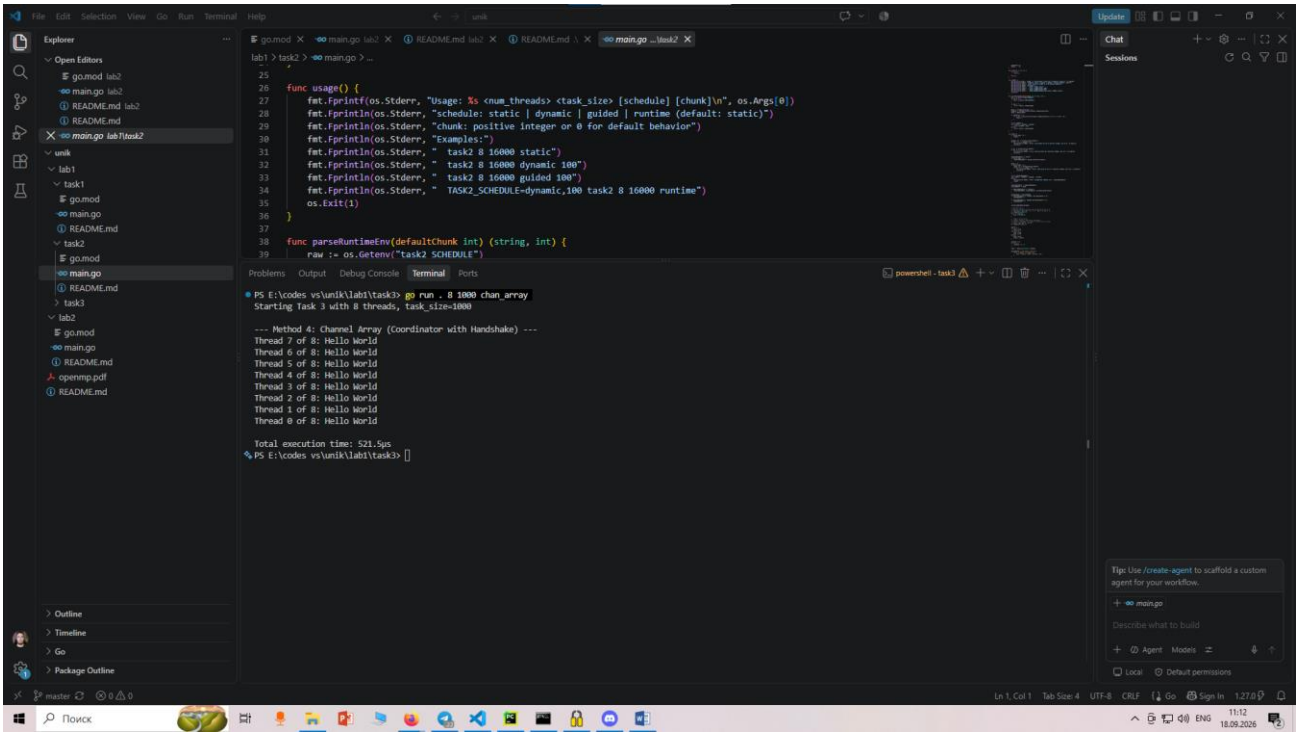


Рисунок 12 — Задача 3: способ 4 (координатор с массивом каналов)

На рисунке 13 показана версия со способом 5 (цепочка sync.WaitGroup). Каждый поток ждёт завершения потока с большим номером; порядок вывода строго обратный.

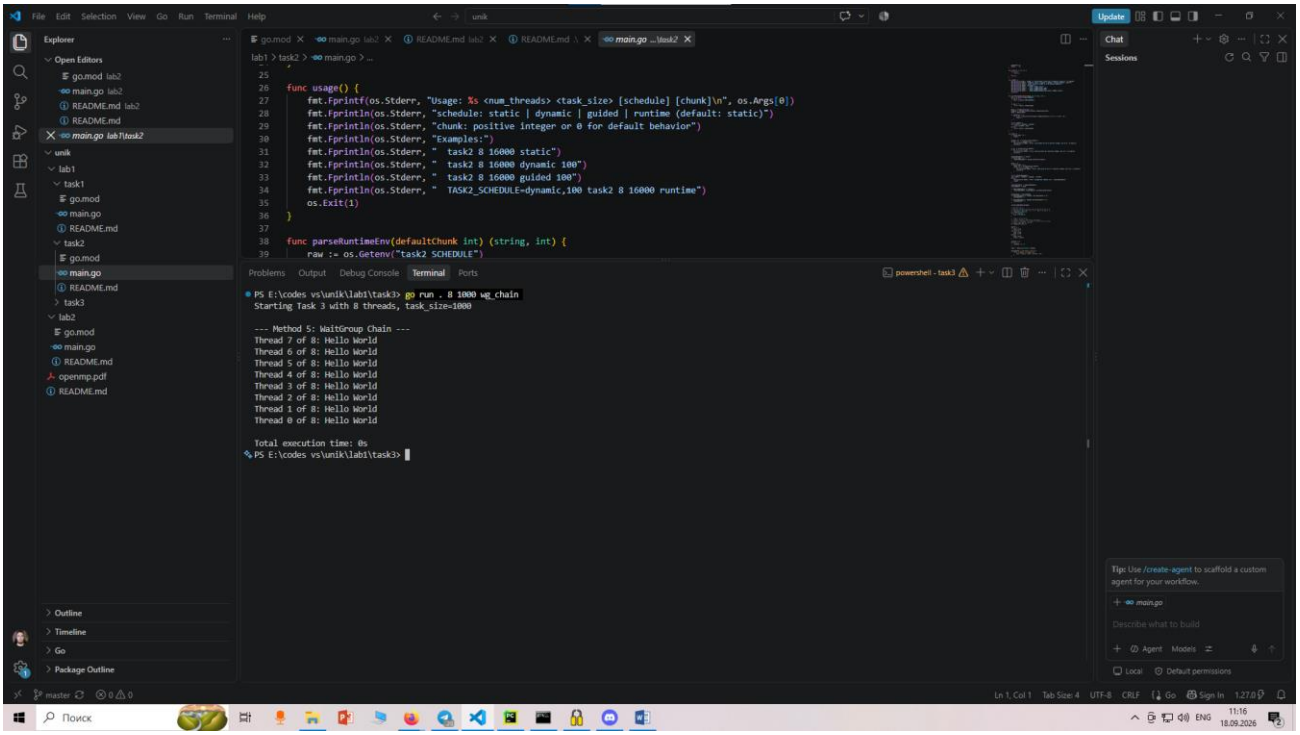


Рисунок 13 — Задача 3: способ 5 (цепочка sync.WaitGroup)

4. Ответы на вопросы, поставленные в задачах

4.1. Вопрос задачи 1: «Всегда ли вывод идентичен? Почему?»

Нет, вывод не всегда идентичен. Потоки выполняются параллельно, и порядок, в котором они достигают оператора вывода, определяется планировщиком, а не программой. Во всех запусках печатается один и тот же набор строк (по одной от каждого из 8 потоков), но последовательность этих строк может меняться. Это подтверждено тремя запусками программы (рисунки 1–3).

4.2. Задача 2: вопрос не поставлен

В задаче 2 вопрос не ставится; требовалось выполнить запуск с 8 потоками при различных типах распределения работ, что подтверждено рисунками 4–8. Ключевое наблюдение: при независимых итерациях результат (контрольная сумма) одинаков для всех типов расписания, различаются только распределение итераций по потокам и время выполнения.

4.3. Вопрос задачи 3: «Существует как минимум 5 способов решения»

Реализовано и продемонстрировано 5 способов фиксации обратного порядка вывода потоков: (1) активное ожидание на атомарном счётчике; (2) условная переменная sync.Cond; (3) цепочка каналов с передачей эстафеты; (4) координатор с массивом каналов и рукопожатием; (5) цепочка sync.WaitGroup. Все пять версий показали корректный порядок 7, 6, …, 0 (рисунки 9–13). Аналогом упорядоченного вывода в OpenMP является директива ordered. Существуют и другие способы (например, искусственные задержки, зависящие от номера потока), однако они не гарантируют порядок и поэтому в работе не использовались.

4.4. Дополнительные вопросы к защите

1. Чем в Go моделируется переменная окружения OMP_NUM_THREADS? Числом потоков, передаваемым через командную строку, и вызовом runtime.GOMAXPROCS(N).
2. В чём различие типов расписания static, dynamic, guided? static делит итерации заранее; dynamic выдаёт блоки из общей очереди по мере освобождения потоков; guided аналогичен dynamic, но размер блока уменьшается по мере убывания оставшейся работы.
3. Что такое состояние гонки и как оно предотвращается? Это неопределённость результата при одновременном доступе потоков к разделяемым данным без синхронизации; предотвращается разделением данных по потокам, мьютексами, атомарными операциями и каналами.
4. Почему результат задачи 2 не зависит от типа расписания? Итерации независимы: каждая пишет только в свой элемент b[i] и читает неизменяемый массив a, поэтому гонки данных отсутствуют.

5. Выводы

1. В задаче 1 реализована параллельная область с идентификацией потоков. Установлено, что без синхронизации порядок вывода потоков недетерминирован: набор строк одинаков во всех запусках, а их последовательность определяется планировщиком.
2. В задаче 2 реализованы и сравнены типы распределения работ static (с блоком и без), dynamic, guided и runtime. Подтверждено, что при независимых итерациях результат не зависит от расписания (контрольная сумма совпала во всех запусках), а различаются балансировка загрузки потоков и время выполнения.
3. В задаче 3 реализовано пять способов синхронизации, фиксирующих обратный порядок вывода потоков, и сравнены их свойства: активное ожидание расходует процессорное время, условные переменные и каналы позволяют ожидать без нагрузки на CPU, координатор даёт наиболее жёсткий контроль порядка.